



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	M Hafizh Muhyiddin
NPM	2428240127
Kelas	SI5C
Topik Tugas 1	Blog
Entitas utama	Artikel
Tautan repositori	https://github.com/hadinnn/si5c_kuis1_mhafizhmuhyiddin
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Blog
Entitas utama (tunggal)	Artikel
Alamat dasar	/articles
Field data	Id, judul, isi, penulis, kategori, dipublikasikan
Field wajib saat POST	judul, isi, penulis, kategori
Query string untuk filter	kategori
Data awal	<pre>{ id: 1, judul: 'Mengenal REST API', isi: 'REST adalah gaya arsitektur ...', penulis: 'Nadia Putri', kategori: 'Teknologi', dipublikasikan: true }, { id: 2, judul: 'Tragedi Dibalik Asap di Palembang', isi: 'Isi artikel tentang tragedi di Palembang', penulis: 'Aldi Wijaya', kategori: 'Tragedi', dipublikasikan: false }</pre>
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-hari]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-hari]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-hari]	Ya	201	Tambah data
4	PUT	/[sumber-hari]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-hari]/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah node -v, npm -v, npm init -y, dan npm install express cors dotenv beserta keluarannya sampai selesai.

```
C:\Users\Hadin\Documents\GitHub\si5c_kuis1_mhafizhmuhyiddin>node -v
v24.16.0

C:\Users\Hadin\Documents\GitHub\si5c_kuis1_mhafizhmuhyiddin>npm -v
11.13.0

C:\Users\Hadin\Documents\GitHub\si5c_kuis1_mhafizhmuhyiddin>npm init -y
Wrote to C:\Users\Hadin\Documents\GitHub\si5c_kuis1_mhafizhmuhyiddin\package.json:

{
  "name": "si5c_kuis1_mhafizhmuhyiddin",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

C:\Users\Hadin\Documents\GitHub\si5c_kuis1_mhafizhmuhyiddin>npm install express
added 68 packages, and audited 69 packages in 3s

28 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\Users\Hadin\Documents\GitHub\si5c_kuis1_mhafizhmuhyiddin>cors dotenv
'cors' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\Hadin\Documents\GitHub\si5c_kuis1_mhafizhmuhyiddin>npm install cors dotenv
added 3 packages, and audited 72 packages in 1s

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Express berfungsi sebagai *framework* web utama Node.js untuk menangani *routing* HTTP dan alur *request-response* aplikasi. Sementara itu, CORS berguna sebagai *middleware* yang mengizinkan aplikasi *frontend* dari *port* atau domain lain melakukan *request* ke server secara aman. Adapun Dotenv bertugas memuat variabel lingkungan dari berkas `.env` ke `process.env`, sehingga informasi sensitif seperti *port* server atau kredensial *database* tersimpan rapi di luar kode utama.

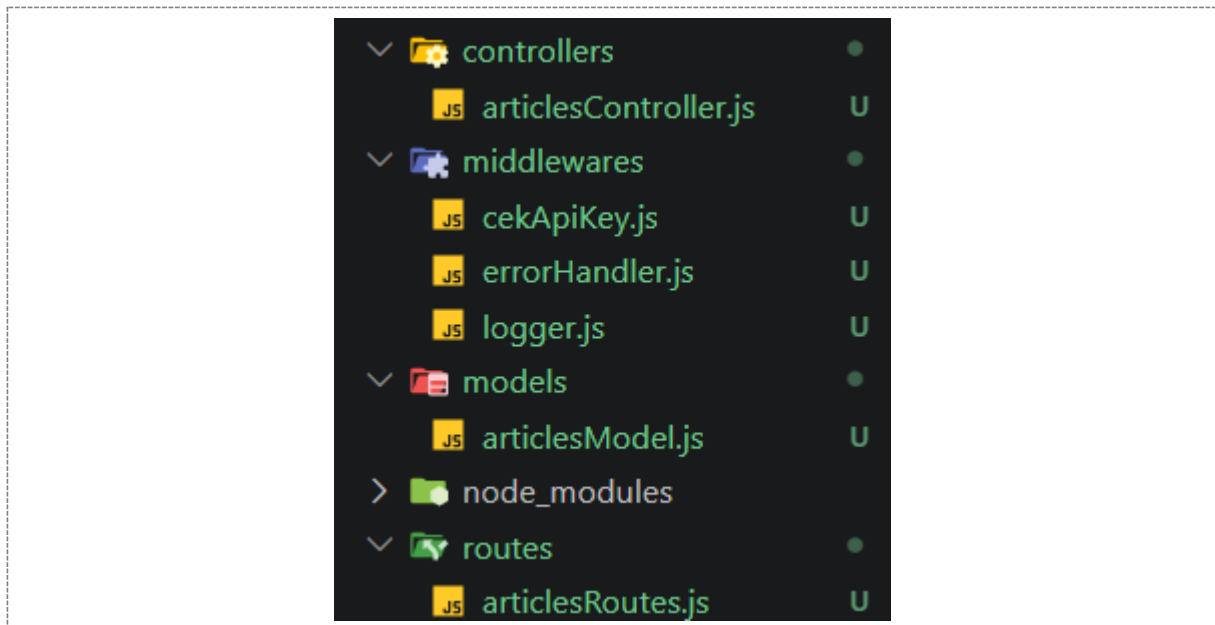
Berdasarkan tampilan terminal, versi Node.js yang terpasang adalah v24.16.0 yang sudah memenuhi syarat minimal Express 5. Versi Node.js harus memenuhi syarat Express 5 karena versi terbaru ini memiliki dukungan bawaan untuk penanganan *error* pada *route handler* berbasis *async/await* secara otomatis. Selain itu, penggunaan versi Node.js yang sesuai memastikan kompatibilitas penuh dengan *package* modern seperti Express, CORS, dan Dotenv, menghindari kecacatan *runtime*, serta menjaga performa dan keamanan sistem backend tetap optimal.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder `routes`, `controllers`, `models`, dan `middlewares` beserta isinya. Gunakan panel *Explorer* editor atau perintah *tree*.



Gambar 2. Struktur folder project

Penjelasan:

Folder `controllers` yang berisi `articlesController.js` bertugas mengolah logika utama aplikasi dalam memproses data artikel dan menyusun respon balasan untuk pengguna. Folder `middlewares` yang memuat `cekApiKey.js`, `errorHandler.js`, dan `logger.js` bertugas melakukan pemeriksaan awal seperti pengecekan kunci akses, pencatatan log, serta penanganan *error*. Folder `models` yang berisi `articlesModel.js` bertugas mengelola struktur data dan berkomunikasi langsung dengan *database* artikel. Sementara itu, folder `routes` yang berisi `articlesRoutes.js` bertugas mengatur jalur atau alamat URL agar setiap *request* yang masuk bisa diarahkan ke fungsi yang sesuai.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

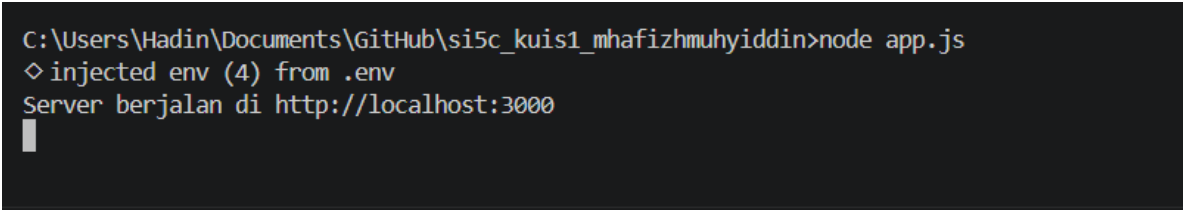
Berkas	Penjelasan Anda
models/articlesModel.js	Berfungsi sebagai tempat penyimpanan dan pengelolaan data artikel sementara yang tersimpan di dalam memori <i>array</i> . Berkas ini menyediakan fungsi-fungsi operasi dasar CRUD untuk membaca, menambah, memperbarui, serta menghapus data artikel. Seluruh fungsi tersebut kemudian di- <i>export</i> agar bisa dipanggil oleh <i>controller</i> saat menangani permintaan dari pengguna.
controllers/articlesController.js	Berfungsi sebagai pengontrol logika bisnis yang menerima permintaan dari pengguna dan menentukan respon yang akan dikirimkan balik. Berkas ini bertugas memvalidasi data masukan, memanggil fungsi pemrosesan data pada <i>articlesModel</i> , serta meneruskan pesan kesalahan ke <i>middleware</i> jika input tidak valid atau data tidak ditemukan. Hasil akhirnya kemudian dikembalikan ke pengguna dalam bentuk respons JSON beserta status HTTP yang sesuai.
routes/articlesRoutes.js	Berkas <i>articlesRoutes.js</i> berfungsi mendefinisikan seluruh <i>endpoint</i> URL beserta metode HTTP yang digunakan untuk mengelola data artikel. Berkas ini menerapkan <i>middleware</i> <i>cekApiKey</i> pada jalur tertentu guna memastikan tindakan penambahan, pembaruan, dan penghapusan data hanya dapat dilakukan oleh permintaan yang memiliki izin. Semua jalur permintaan tersebut kemudian dihubungkan langsung ke fungsi penanganan yang sesuai di dalam <i>articlesController</i> .
middlewares/logger.js	mencatat riwayat setiap permintaan HTTP yang masuk ke server secara <i>real-time</i> . Berkas ini mencetak informasi berupa stempel waktu (<i>timestamp</i>), metode HTTP, dan URL yang diakses pengguna langsung ke terminal. Setelah proses pencatatan selesai, fungsi <i>next()</i> dipanggil agar aliran permintaan dapat dilanjutkan ke proses atau <i>middleware</i> berikutnya.
middlewares/cekApiKey.js	<i>middleware</i> keamanan untuk menguji validitas hak akses pengguna melalui <i>header</i> <i>x-api-key</i> . Berkas ini membandingkan nilai kunci yang dikirim oleh pengguna dengan kunci rahasia yang tersimpan pada variabel lingkungan <i>process.env.API_KEY</i> . Jika kunci sesuai, permintaan akan diteruskan ke proses berikutnya melalui fungsi <i>next()</i> , sedangkan jika tidak sesuai, server akan langsung menolak akses dengan respon bernilai status 401.

Berkas	Penjelasan Anda
middlewares/errorHandler.js	berfungsi sebagai penanganan kesalahan terpusat sekaligus penyedia fungsi bantuan (<i>helper</i>) untuk membuat pesan <i>error</i> secara terstruktur. Berkas ini bertugas menangkap rute yang tidak terdaftar, kesalahan sintaks JSON dari pengguna, serta <i>error</i> internal server (500) agar aplikasi tidak mengalami <i>crash</i> . Setiap <i>error</i> yang terjadi kemudian diolah dan dikembalikan dalam bentuk respons JSON dengan pesan serta kode status HTTP yang rapi dan konsisten.
app.js	Berkas app.js berfungsi sebagai titik masuk utama (<i>entry point</i>) dan pusat konfigurasi dari seluruh aplikasi Express. Berkas ini bertugas memuat variabel lingkungan, memasang <i>middleware</i> global seperti CORS dan <i>logger</i> , mengisikan rute /articles, serta mengatur penanganan <i>error</i> terpusat. Terakhir, app.js menjalankan server HTTP sehingga aplikasi dapat mulai menerima dan merespons permintaan pada <i>port</i> yang ditentukan.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



```
C:\Users\Hadin\Documents\GitHub\si5c_kuis1_mhafizhmuhyiddin>node app.js
◇ injected env (4) from .env
Server berjalan di http://localhost:3000
```

Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor *port* berasal dari nilai variabel `PORT` yang dibaca dari berkas `.env` melalui `process.env.PORT`. Jika variabel tersebut tidak diatur di dalam berkas `.env`, aplikasi secara otomatis akan menggunakan nomor *port* cadangan yaitu 3000 melalui operator logika `|| 3000`.

Jika *port* yang ditentukan ternyata sudah dipakai oleh aplikasi lain, perintah `app.listen()` akan gagal mendaftarkan alamat server. Kondisi ini akan menyebabkan Node.js mengeluarkan pesan *error* `EADDRINUSE (Error: address already in use)` dan membuat server Express Anda langsung terhenti (*crash*) serta tidak bisa dijalankan.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

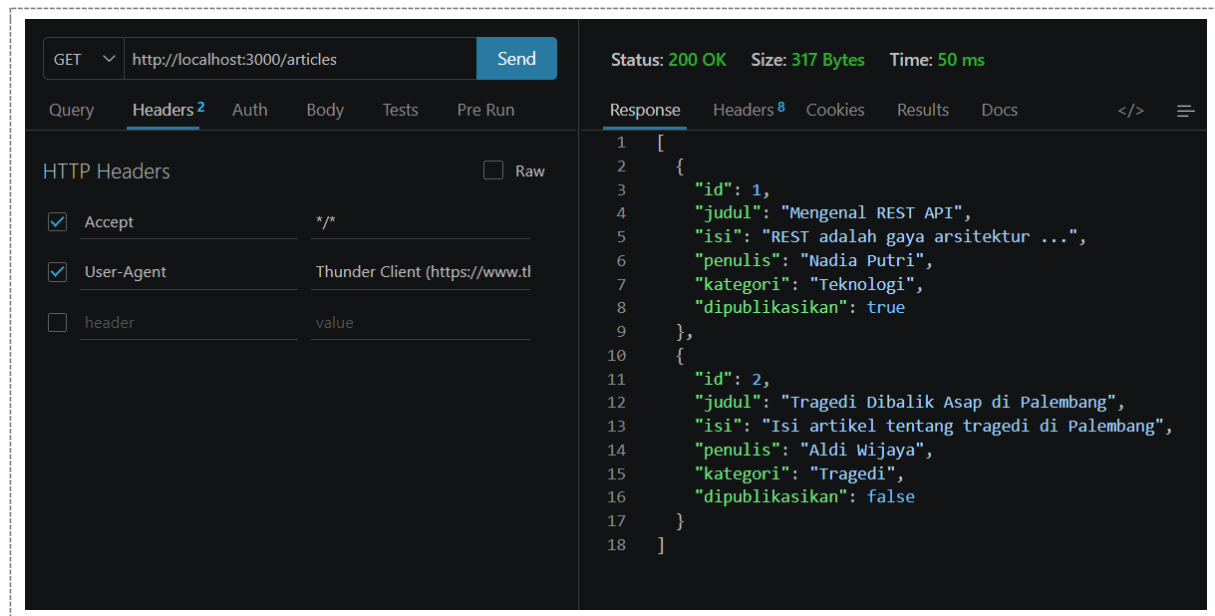
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /articles)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /articles
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	<pre> 200 OK [{ "id": 1, "judul": "Menenal REST API", "isi": "REST adalah gaya arsitektur ...", "penulis": "Nadia Putri", "kategori": "Teknologi", "dipublikasikan": true }, { "id": 2, "judul": "Tragedi Dibalik Asap di Palembang", "isi": "Isi artikel tentang tragedi di Palembang", "penulis": "Aldi Wijaya", "kategori": "Tragedi", "dipublikasikan": false }]</pre>
Lulus?	☒ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

Penjelasan:

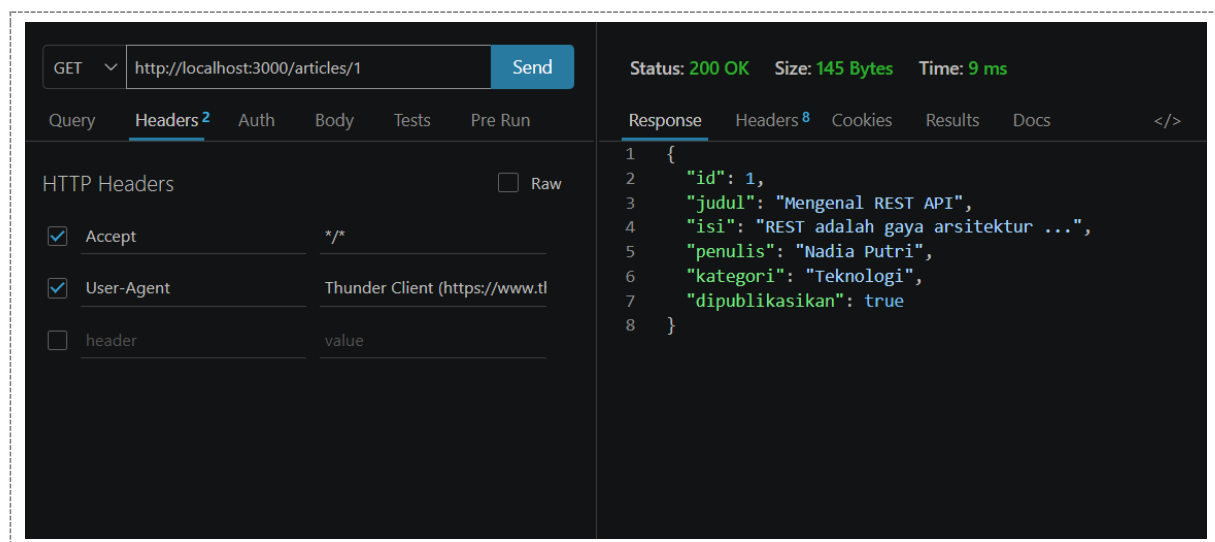
Kode status yang muncul adalah **200 OK**, yang menandakan bahwa permintaan HTTP GET berhasil diproses oleh server dan data berhasil dikembalikan tanpa kendala. *Endpoint* ini memiliki kemampuan untuk mengambil serta menampilkan seluruh daftar artikel yang tersimpan di dalam aplikasi, sekaligus mendukung fitur pemfilteran data jika terdapat parameter *query* kategori yang dikirimkan.

Urutan pemanggilannya dimulai dari *route* `router.get('/')` pada berkas `articlesRoutes.js` (yang dipetakan dari jalur `/articles` pada `app.js`). Selanjutnya, *route* tersebut memanggil fungsi *controller* `getAll` pada berkas `articlesController.js`. Terakhir, fungsi *controller* tersebut menjalankan fungsi *model* `getAll` pada berkas `articlesModel.js` untuk mengambil data artikel dari *array* dan mengembalikannya ke pengguna dalam format JSON.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (<i>request</i>)	GET /articles/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK; { "id": 1, "judul": "Menenal REST API", "isi": "REST adalah gaya arsitektur ...", "penulis": "Nadia Putri", "kategori": "Teknologi", "dipublikasikan": true }
Lulus?	[✓] Ya [] Tidak



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

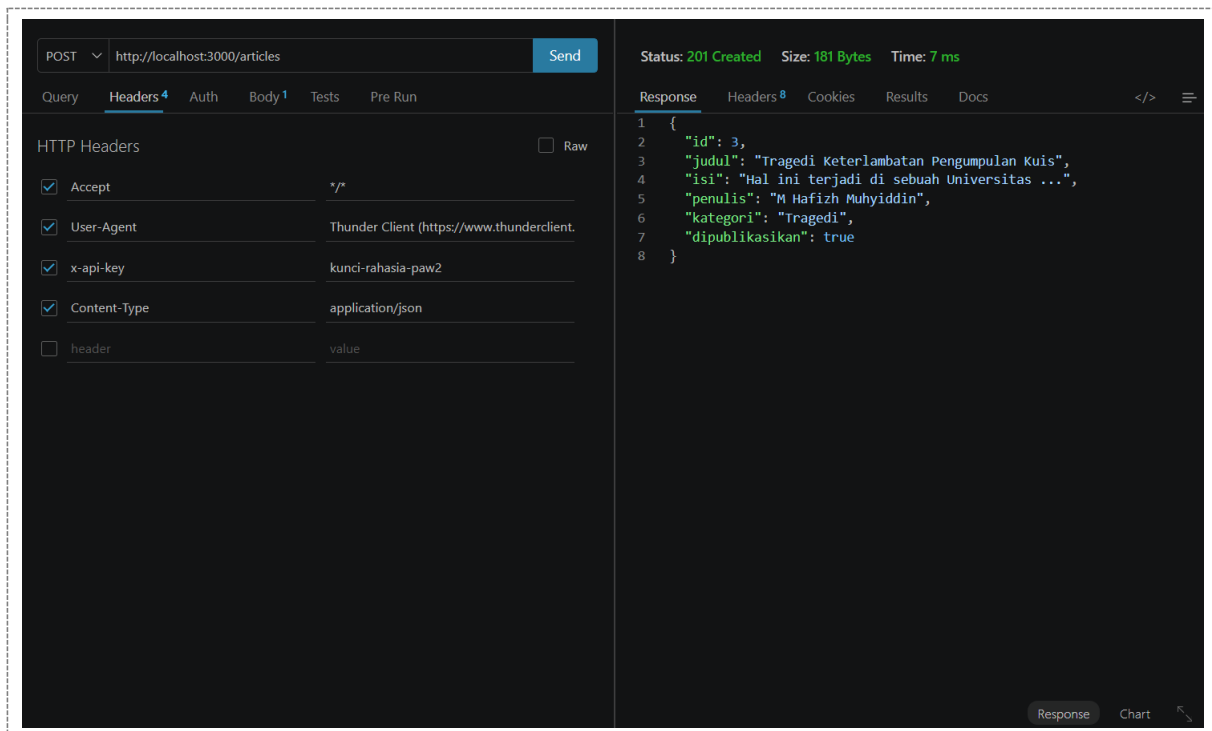
Nilai ID dari alamat URL diambil melalui `req.params.id` karena Express secara otomatis membaca parameter dinamis `:id` yang didefinisikan pada rute `/articles/:id`. Saat pengguna mengakses `http://localhost:3000/articles/1`, Express menangkap bagian ujung URL tersebut dan memasukkannya ke dalam objek `req.params`.

Nilai yang diambil dari URL tersebut selalu bertipe data *string* (misalnya '1'), sehingga perlu diubah menjadi angka menggunakan `parseInt()`. Pengubahan ini wajib dilakukan agar tipe datanya cocok dengan ID di dalam `articlesModel` yang bertipe *number*, sehingga proses pencarian data menggunakan perbandingan ketat (`===`) dapat menemukan artikel yang sesuai.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /articles
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "judul": "Tragedi Keterlambatan Pengumpulan Kuis", "isi": "Hal ini terjadi di sebuah Universitas ...", "penulis": "M Hafizh Muhyiddin", "kategori": "Tragedi" }
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created; { "id": 3, "judul": "Tragedi Keterlambatan Pengumpulan Kuis", "isi": "Hal ini terjadi di sebuah Universitas ...", "penulis": "M Hafizh Muhyiddin", "kategori": "Tragedi", "dipublikasikan": true }
Lulus?	☒ Ya ☐ Tidak



Gambar 6. Hasil POST data baru

Penjelasan:

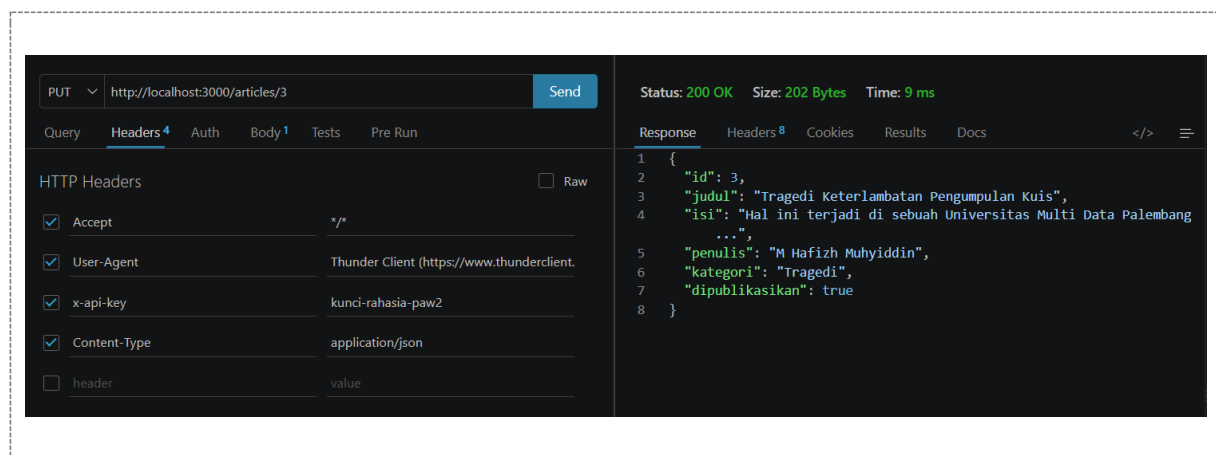
Header `x-api-key` berperan sebagai otentikasi untuk memastikan pengguna memiliki hak akses sebelum menambahkan artikel baru, di mana pemeriksaan ini dilakukan oleh *middleware* `cekApiKey.js`. Sementara itu, `express.json()` bertugas menguraikan (*parse*) data berformat JSON dari *body* permintaan agar bisa diakses secara langsung sebagai objek JavaScript melalui `req.body` di dalam *controller*.

Penggunaan kode status **201 Created** dipilih (bukan 200 OK) karena operasi HTTP POST ini berhasil membuat dan menyimpan sumber daya (*resource*) data artikel baru di server. Dalam standar REST API, kode status 201 secara spesifik menandakan keberhasilan pembuatan data baru, sedangkan kode status 200 OK umumnya dipakai untuk respon sukses yang sifatnya umum seperti membaca (*GET*) atau memperbarui data.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /articles/3
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "isi": "Hal ini terjadi di sebuah Universitas Multi Data Palembang ..." }
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK; { "id": 3, "judul": "Tragedi Keterlambatan Pengumpulan Kuis", "isi": "Hal ini terjadi di sebuah Universitas Multi Data Palembang ...", "penulis": "M Hafizh Muhyiddin", "kategori": "Tragedi", "dipublikasikan": true }
Lulus?	☒ Ya ☐ Tidak



Gambar 7. Hasil PUT mengubah data

Penjelasan:

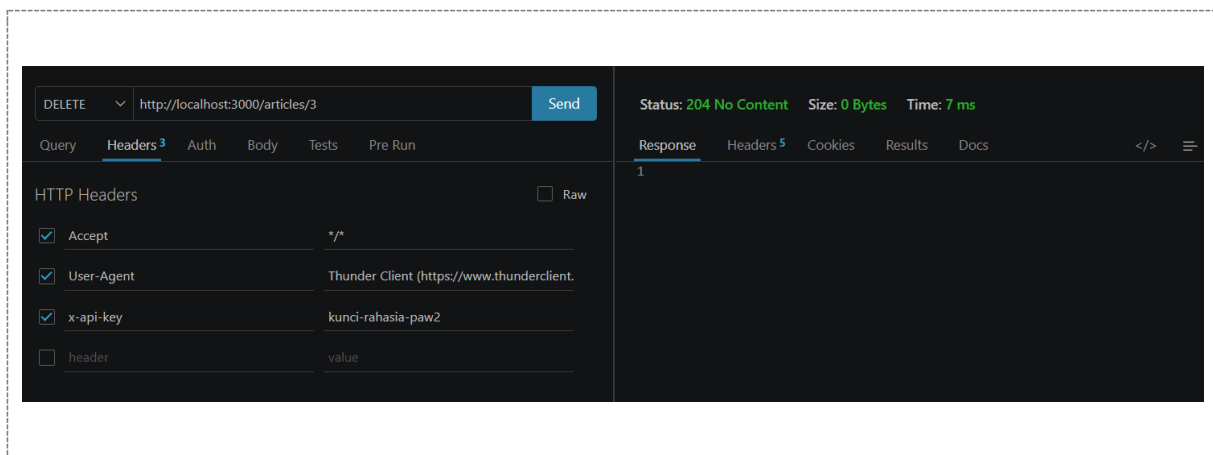
Field yang mengalami perubahan adalah isi, di mana teksnya diperbarui dengan menambahkan keterangan nama kampus menjadi *"Hal ini terjadi di sebuah Universitas Multi Data Palembang ..."*. Sementara itu, *field* lainnya seperti id, judul, penulis, kategori, dan dipublikasikan nilainya tetap sama dan tidak mengalami perubahan.

Proses eksekusi perubahannya dilakukan oleh layer **Model** (articlesModel.js), tepatnya pada fungsi update(). Layer Model ini menggabungkan data lama dengan data baru yang dikirimkan dari *controller*, lalu memperbarui objek artikel di dalam memori *array* berdasarkan ID yang dituju tanpa mengubah nilai ID aslinya.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (<i>request</i>)	DELETE /articles/3
Header	x-api-key: kunci-rahasia-paw2
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content
Lulus?	[✓] Ya [] Tidak



Gambar 8. Hasil DELETE data

Penjelasan:

Kode status **204 No Content** berarti permintaan HTTP DELETE telah berhasil diproses oleh server, tetapi tidak ada data yang perlu dikembalikan kepada pengguna. Respons tidak berisi apa pun (*Size: 0 Bytes*) karena data artikel yang dimaksud telah berhasil dihapus dari memori server. Selain itu, pada berkas `articlesController.js`, fungsi `remove` menggunakan perintah `res.status(204).send()`, yang sesuai standar HTTP memang bertugas mengirimkan respon keberhasilan tanpa menyertakan isi *body* pesan.

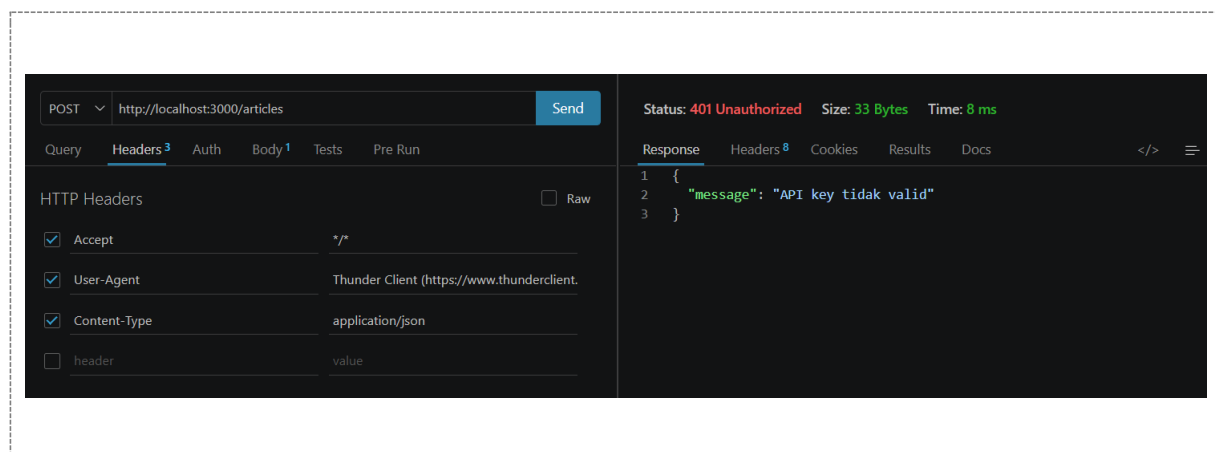
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada **screenshot**: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /articles (tanpa x-api-key)
Header	Content-Type: application/json
Body	<pre>{ "judul": "Tragedi Keterlambatan Pengumpulan Kuis", "isi": "Berita sedih banget ini", "penulis": "M Hafizh Muhyiddin", "kategori": "Tragedi" }</pre>
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized; { "message": "API key tidak valid" }
Lulus?	[✓] Ya [] Tidak



Gambar 9. POST tanpa API key ditolak

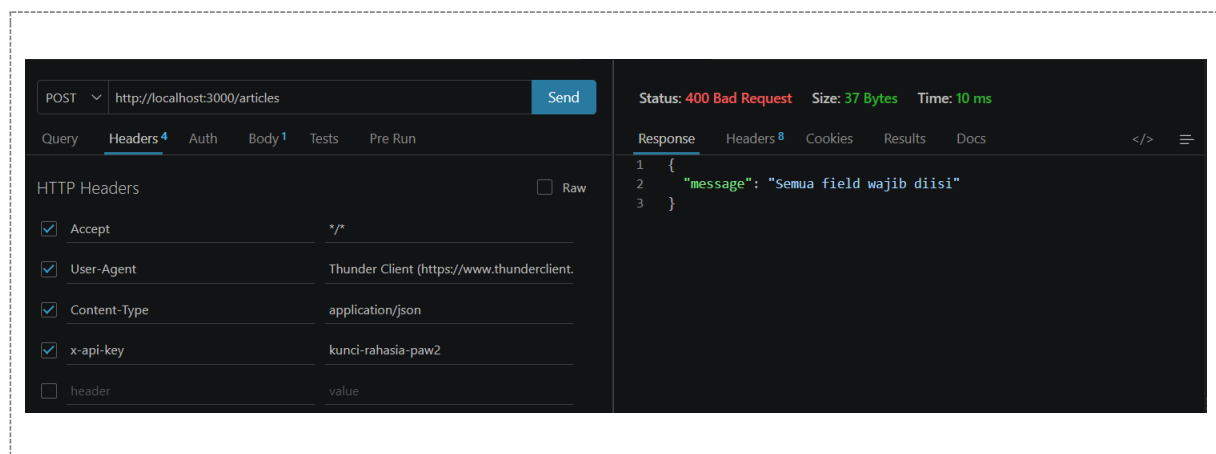
Penjelasan:

Permintaan ini ditolak oleh *middleware* cekApiKey (di berkas cekApiKey.js) karena nilai *header* x-api-key tidak dikirimkan atau tidak sesuai dengan kunci rahasia di server. *Controller* tidak sempat dijalankan karena *middleware* cekApiKey langsung menghentikan aliran permintaan dengan mengembalikan respons status **401 Unauthorized** tanpa memanggil fungsi next(). Karena next() tidak pernah dipanggil, Express memutuskan rantai proses sehingga eksekusi kode terhenti dan tidak pernah diteruskan ke fungsi *controller* (articlesController.create).

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /articles
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "judul": "Tragedi Keterlambatan Pengumpulan Kuis", "isi": "Berita sedih banget ini", "kategori": "Tragedi" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request; { "message": "Semua field wajib diisi" }
Lulus?	[✓] Ya [] Tidak



Gambar 10. POST dengan body tidak lengkap

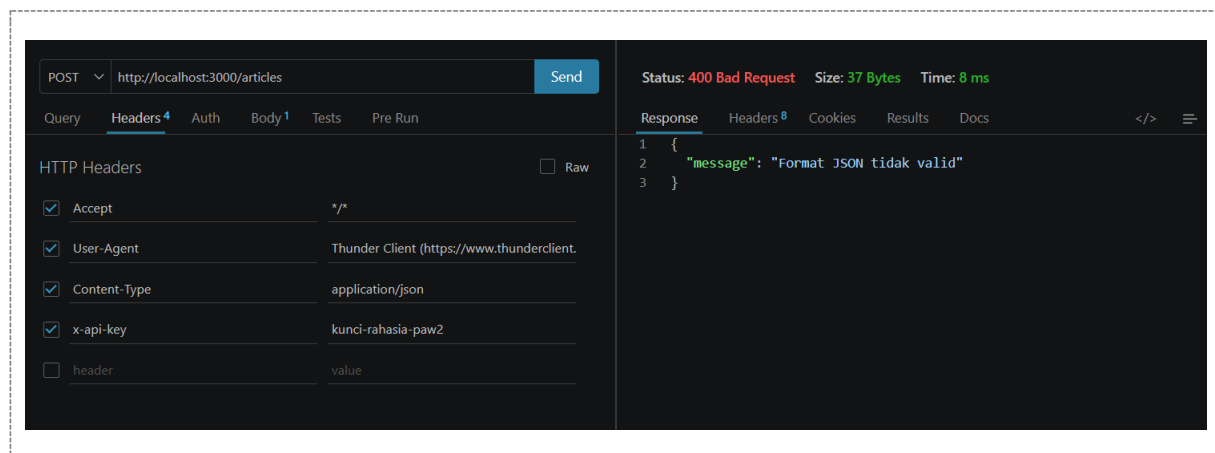
Penjelasan:

Validasi ini ditulis di layer **Controller**, tepatnya pada fungsi create di dalam berkas `articlesController.js`. Kode status yang digunakan adalah **400 Bad Request** karena kesalahan berasal dari pihak pengguna (*client error*), di mana pengguna tidak melengkapi atau mengosongkan *field* wajib yang dibutuhkan oleh server untuk memproses pembuatan data artikel.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (request)	POST /articles
Header	x-api-key: kunci-rahasia-paw2 Content-Type: application/json
Body	{ "judul": "Tragedi Keterlambatan Pengumpulan Kuis", "isi": "Berita sedih banget ini", "penulis": "M Hafizh Muhyiddin", "kategori": "Tragedi", }
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request; { "message": "Format JSON tidak valid" }
Lulus?	[✓] Ya [] Tidak



Gambar 11. POST dengan JSON rusak

Penjelasan:

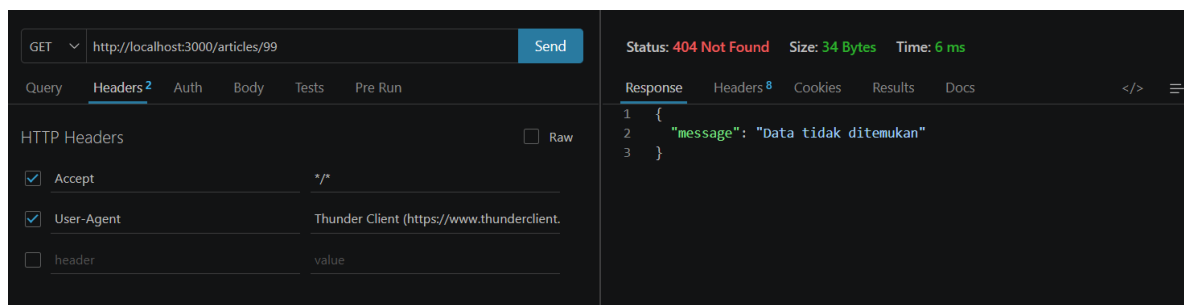
JSON yang rusak dideteksi oleh *middleware* `express.json()` saat mencoba menguraikan data *body* permintaan yang masuk. Ketika terjadi kesalahan sintaks, `express.json()` secara otomatis membangkitkan objek *error* bertipe `'entity.parse.failed'` dan meneruskannya ke rantai penanganan *error*.

Selanjutnya, *middleware* `errorHandler.js` menangkap *error* tersebut melalui kondisi pengecekan `if (err.type === 'entity.parse.failed')`. Fungsi ini menghentikan *stack trace error* bawaan Node.js dan menggantinya dengan respon JSON yang rapi berupa `{ message: 'Format JSON tidak valid' }` beserta kode status **400 Bad Request**.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /articles/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found; { "message": "Data tidak ditemukan" }
Lulus?	[✓] Ya [] Tidak



Gambar 12. GET ID tidak ada

Penjelasan:

Permintaan GET /articles/99 awalnya dicocokkan dengan rute /articles/:id pada articlesRoutes.js, lalu diteruskan ke fungsi getByld di dalam articlesController.js. Controller kemudian memanggil fungsi model articlesModel.getByld(99) untuk mencari data artikel terkait di dalam memori *array*.

Karena artikel dengan ID 99 tidak ada di dalam *array* (menghasilkan nilai undefined), layer **Controller** (articlesController.js) mendeteksi kondisi tersebut dan secara langsung membuat serta mengirimkan respons status **404 Not Found** dengan pesan {"message": "Data tidak ditemukan"}.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200		
T02	Menampilkan satu data berdasarkan ID	200		
T03	Menambah data baru	201		
T04	Mengubah data	200		
T05	Menghapus data	204		
N01	Menambah data tanpa API key	401		
N02	Menambah data dengan body tidak lengkap	400		
N03	Mengirim JSON yang rusak	400		
N04	Mengambil ID yang tidak ada	404		

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/hadinnn/si5c_kuis1_mhafizhm_uhyiddin
Pesan commit terakhir	Menambahkan MVC untuk Topik Blog
Folder routes, controllers, models, middlewares ada	☐ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☐ Ya ☐ Tidak
.env.example ada	☐ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

- **Route** : Tugasnya mengatur alamat URL sama metode HTTP yang masuk, lalu ngarahin permintaan itu ke fungsi controller yang pas. *Contoh (articlesRoutes.js)*: `router.get('/', articlesController.getAll);`
- **Controller** : Ini bagian otaknya yang mengerjakan logika bisnis, seperti ngecek validasi data masukan pengguna, panggil model, terus mengirim balikan respon HTTP ke pengguna. *Contoh (articlesController.js)*: `const articles = articlesModel.getAll();`
- **Model** : Fokusnya cuma urusan data saja, seperti menyimpan, mengambil, atau mengubah data artikel di dalam *array* tanpa perlu tahu urusan HTTP sama sekali. *Contoh (articlesModel.js)*: `exports.getAll = () => articles;`

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Keuntungan utamanya bikin kodingan kita jauh lebih rapi dan format respon *error* di seluruh aplikasi jadi seragam. Kita juga nggak perlu repot nulis `try-catch` atau `res.status()` berulang kali di tiap *route* dan *controller*, soalnya semua *error* dari *middleware* maupun sistem bakal otomatis ditangani di satu tempat.

Buktinya kelihatan banget pas pengujian **format JSON rusak** kemarin. Tanpa kita nambahin penanganan *error* manual di `articlesController.js`, `middleware errorHandler.js` langsung otomatis nangkep *error* dari `express.json()` terus ngeluarin respon rapi berstatus **400 Bad Request** dengan pesan `{"message": "Format JSON tidak valid"}`.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [...] sudah diganti dengan topik Tugas 1 saya	
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	
4	Identitas saya (nama atau NIM) muncul pada data uji POST	
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	
6	Tabel rekapitulasi sesuai dengan tangkapan layar	
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100